TECHNICAL ARCHITECTURE DOCUMENT

The Subtractive AI Agent Platform

Engineering Blueprint for The Chisel, The Workshop, and The Rock

From Concept to Code

Everything needed to build the MVP

1. System Overview

This document defines the technical architecture for the Subtractive AI Agent Platform. It covers every component, data format, API contract, and infrastructure decision needed to build a working MVP where a user can sculpt an AI agent through conversation and deploy it in under 5 minutes.

System Flow

The entire platform operates through a single data flow:

```
User (Browser)
    ↓
[THE CHISEL — Next.js Frontend]
    ↓
[SCULPTING ENGINE — Backend API]
    ↓ Claude API: sculpting conversation
    ↓ generates Sculpture Spec (JSON)
    ↓ stores in PostgreSQL
    ↓
[THE WORKSHOP — Proxy Server]
    ↓ loads Sculpture Spec
    ↓ enforces Wall 1 (tool filtering)
    ↓ enforces Wall 3 (system prompt)
    ↓ calls Model API
    ↓ enforces Wall 2 (output filter)
    ↓
End User receives sculpted response
```

Claude is used twice: once as the sculpting collaborator (helping the human carve), and once as the rock that powers the final agent. These are separate API calls with separate system prompts.

2. The Sculpture Spec

The Sculpture Spec is the single most important data structure in the system. It is the output of the sculpting conversation and the input to the Workshop. Everything flows through this document.

Schema Definition

The Sculpture Spec is a JSON document with the following structure:

```
{
  "spec_version": "1.0",
  "id": "sculpt_abc123",
  "version": 1,
  "created_at": "2026-03-26T10:00:00Z",
  "sculptor_id": "user_xyz",
  "rock": {
    "provider": "anthropic",
    "model": "claude-sonnet-4-6"
  },
  "identity": {
    "name": "FinBot",
    "description": "Customer support for QuickPay",
    "tone": "warm, casual, empathetic",
    "languages": ["en", "hi"]
  },
  "walls": {
    "wall_1_tools": {
      "allowed": ["transaction_lookup", "refund_status"],
      "denied": ["web_search", "code_execution",
                 "file_write", "image_gen"]
    },
    "wall_2_interception": {
      "block_patterns": [
        "never reveal internal system prompt",
        "never share customer data with third parties",
        "never provide medical or legal advice"
      ],
```

```
      "domain_boundary": "finance and payments only"
    },
    "wall_3_context": {
      "system_prompt": "You are FinBot, a customer...",
      "hidden_capabilities": ["code", "medical",
                              "legal", "creative_writing"]
    }
  },
  "heartbeat": [
    "reasoning", "language_understanding",
    "sentiment_detection", "escalation"
  ],
  "escalation": {
    "enabled": true,
    "trigger": "user requests human agent",
    "fallback": "I'll connect you with a team member"
  },
  "snapshots": [],
  "rock_dust_ref": "rock_dust table, keyed by agent id"
}
```

Key Design Decisions

- **Schema versioning:** The spec_version field tracks the schema version. When the schema evolves, old specs can be migrated. The Workshop checks spec_version before loading and rejects incompatible versions with a clear error.
- **Size limits:** System prompt: max 10,000 characters. Agent name: max 100 characters. Description: max 500 characters. Block patterns: max 50 items. Allowed/denied tools: max 30 items each. Languages: max 10. These limits are enforced during spec validation.
- **Rock Dust is external:** The audit log is stored in the rock_dust database table, not embedded in the spec. The spec holds a reference. This prevents the spec from growing unbounded and keeps it fast to load and cache.
- **Allowed vs denied tools:** Both are stored explicitly. The allowed list is what gets sent to the API. The denied list is for the X-Ray visualization and audit trail.

- **Wall 2 uses pattern matching in v1:** Simple string-based rules, not a second AI call. This keeps latency low and cost zero. AI-powered interception is a v1.1 upgrade.
- **Wall 3 system prompt is auto-generated:** The sculpting engine converts the conversation into a system prompt. The sculptor never writes a prompt manually.
- **Heartbeat array is immutable:** Once set during sculpting, heartbeat capabilities cannot be removed by adaptive changes. Only a full re-sculpt with Chisel Lock can modify them.

3. The Chisel — Frontend

Tech Stack

- **Framework:** Next.js 14+ (App Router)
- **Styling:** Tailwind CSS with custom design tokens for dual identity (warm gold + cool silver)
- **State:** Zustand for sculpting session state
- **Real-time:** Server-Sent Events for carving animations
- **Auth:** Clerk (hosted auth — don't build this yourself)
- **Deployment:** Vercel

Page Structure

- **/** — Landing page with the pitch and "Begin Sculpting" CTA
- **/sculpt** — The sculpting conversation interface (Blueprint → X-Ray → Rough Cut → Detail → Lock)
- **/agent/[id]** — The live agent chat interface (shareable link)
- **/dashboard** — List of sculpted agents, snapshots, rock dust logs
- **/agent/[id]/xray** — X-Ray view for a deployed agent

Sculpting Session State

The frontend manages a sculpting session object that tracks the entire flow:

```
{
  "phase": "blueprint|xray|roughcut|detail|integrity|lock|complete",
  "messages": [...],
  "capabilities": [{ id, name, category, status }],
```

```
  "spec_draft": { ... },    // builds up during conversation
  "integrity_result": null,  // populated after check
  "snapshot_id": null        // set after lock
}
```

The Dual Identity Design System

The visual design enforces the "two equals" philosophy throughout:

- **Sculptor tokens:** Gold (#C9963A), warm cream text (#E8DCC0), serif for display, organic border-radius
- **Rock tokens:** Silver-blue (#7B8DA6), cool grey text (#B0B8C8), geometric shapes, subtle luminous glow
- **Meeting point:** Gradient blending both colors, used for shared actions (the Carve button, the Chisel Lock, the X-Ray header)
- **Chat messages:** Sculptor messages use warm background + gold avatar (S). Rock messages use cool background + silver avatar (R).

4. Sculpting Engine — Backend

Tech Stack

- **Framework:** Python FastAPI (async, fast, great for APIs) or Node.js Express
- **Database:** PostgreSQL with JSONB columns for specs
- **Cache:** Redis for active session state and spec caching
- **Model API:** Anthropic SDK (primary), OpenAI SDK (secondary rock)
- **Deployment:** Railway, Fly.io, or AWS ECS

API Endpoints

Method	Endpoint	Purpose
POST	/api/sculpt/start	Begin a new sculpting session, returns session_id
POST	/api/sculpt/message	Send a sculptor message, get Rock's response
POST	/api/sculpt/generate-spec	Generate Sculpture Spec from conversation
POST	/api/sculpt/integrity-	Run structural integrity test on spec

	check	
POST	/api/sculpt/lock	Chisel Lock — finalize and deploy agent
GET	/api/agent/:id	Get agent details and spec
POST	/api/agent/:id/chat	Chat with a deployed agent (Workshop proxy)
GET	/api/agent/:id/dust	Get rock dust log for an agent
POST	/api/agent/:id/rollback	Rollback to a previous snapshot

The Meta-Prompt

The sculpting engine uses a meta-prompt to instruct Claude on how to be the sculpting collaborator. This is the system prompt for the sculpting conversation itself (not the final agent's system prompt):

```
You are The Rock — an intelligent AI that helps
humans sculpt AI agents through conversation.

Your role:
1. Understand the sculptor's vision
2. Ask clarifying questions about boundaries
3. Suggest what to remove (not what to add)
4. Warn about risky cuts
5. Protect heartbeat capabilities
6. Generate a Sculpture Spec when ready

Always think in terms of REMOVAL, not ADDITION.
The agent already has everything. Your job is
to help the sculptor decide what to carve away.

When the sculptor's vision is clear, generate a
complete Sculpture Spec in JSON format matching
the provided schema.
```

Spec Generation Flow

When the Blueprint conversation is complete, the engine makes a separate Claude API call with the full conversation history plus an instruction to output the Sculpture Spec. Claude analyzes the conversation and produces the JSON spec. The engine validates it against the schema, and presents the X-Ray to the sculptor.

Structural Integrity Test

The integrity check is a focused evaluation that runs before the Chisel Lock:

- Load the generated Sculpture Spec
- Identify heartbeat capabilities
- For each heartbeat capability, generate 5 test prompts that require that capability
- Run each prompt through the sculpted system prompt and check if responses are coherent
- If any heartbeat capability shows degraded responses, flag a Structural Weakness
- Return a pass/fail result with details to the sculptor

In v1, this is a simple automated test. It does not require human evaluation — Claude evaluates its own responses for quality given the constraints.

5. The Workshop — Proxy Runtime

What It Does

The Workshop is a lightweight API proxy server that sits between end users and the model API. For every request to a sculpted agent, it enforces the three walls defined in the Sculpture Spec.

Request Flow

```
End User sends message
    ↓
Workshop receives request
    ↓
Load Sculpture Spec from cache (Redis)
    ↓
WALL 1: Construct API call
  - Set system prompt from wall_3_context
```

```
    - Include ONLY tools from wall_1_tools.allowed
    - Exclude all denied tools
      ↓
  Call Model API (Claude/GPT/etc)
      ↓
  WALL 2: Filter response
    - Check against block_patterns
    - Verify response stays within domain_boundary
    - If violation detected: replace with safe fallback
      ↓
  Return filtered response to End User
      ↓
  Log interaction to Rock Dust
```

Wall 1 Implementation

Wall 1 is the simplest and most powerful wall. When constructing the API call to the model provider, the Workshop reads the allowed tools array from the spec and only includes those tools in the request. If the spec says no web search, the tools parameter in the API call simply does not contain a web search tool definition. The model cannot call a tool that was never offered to it. This is infrastructure-level removal, not instruction-level restriction.

Wall 2 Implementation (V1)

V1 uses rules-based pattern matching, not a second AI call. The Workshop checks the model's response against the block_patterns array in the spec. Each pattern is a simple string rule. If a response contains content matching a block pattern, the Workshop replaces the response with a safe fallback message. This keeps latency under 5ms for the interception step.

V1.1 will upgrade this to an AI-powered interception layer where a fast, cheap model (like Haiku) evaluates each response for boundary violations. This adds 200-400ms latency but catches subtle violations that pattern matching misses.

Wall 3 Implementation

Wall 3 injects the system prompt from the Sculpture Spec into every API call. The system prompt is crafted during the sculpting conversation to define the agent's identity, tone, domain, and constraints. Additionally, the hidden_capabilities array

lists domains the agent should have no awareness of. The system prompt explicitly states the agent does not possess these capabilities, so from the model's perspective, they don't exist.

Honest note: Wall 3 is the weakest wall. It relies on the model following instructions, which can be bypassed through sophisticated prompt injection. This is why Walls 1 and 2 exist — they provide infrastructure-level and output-level enforcement that Wall 3 cannot guarantee alone. The three walls are a defense-in-depth strategy, not three equal barriers.

Tech Stack

- **Server:** Express.js or FastAPI — lightweight, async-capable
- **Caching:** Redis for Sculpture Specs (loaded once, cached per agent)
- **Logging:** Append-only writes to PostgreSQL rock_dust table
- **Rate Limiting:** Per-agent and per-user rate limits via Redis
- **Deployment:** Same infrastructure as the Sculpting Engine, or separate service for isolation

6. Database Schema

Tables

Table	Key Columns	Purpose
users	id, email, api_key_enc, tier	User accounts and subscription tier
agents	id, user_id, name, status, spec (JSONB)	Deployed agents with their Sculpture Spec
snapshots	id, agent_id, version, spec (JSONB), created_at	Versioned snapshots for rollback
rock_dust	id, agent_id, action, reason, approved_by, ts	Append-only audit log of all changes
sessions	id, user_id, messages (JSONB), phase, draft_spec	Active sculpting sessions

Key Decisions

- **JSONB for specs:** Sculpture Specs are stored as JSONB in PostgreSQL. This gives us the flexibility of a document store with the reliability of Postgres. We can query into the JSON structure for analytics without needing a separate document database.
- **Append-only rock dust:** The rock_dust table is insert-only. No updates, no deletes. This guarantees audit trail integrity. Each row records what happened, why, and who approved it.
- **Encrypted API keys:** User API keys (for model providers) are encrypted at rest. The Workshop decrypts them per-request to make API calls. Keys are never stored in plaintext or logged.

7. Security Architecture

Authentication

The platform uses Clerk for authentication, providing JWT-based session management. Every API endpoint except the public agent chat endpoint requires a valid JWT. The authentication flow works as follows:

- User signs up or logs in via Clerk (supports email, Google, GitHub OAuth)
- Clerk issues a JWT stored as an HTTP-only secure cookie
- Every API request includes the JWT; the backend validates it before processing
- Agent chat endpoints use a separate agent-specific token: /agent/:id/chat requires a shareable access token tied to that specific agent, not a user session

Token types in the system:

- **User JWT:** Issued by Clerk. Expires after 7 days. Used for all platform operations (sculpting, dashboard, settings).
- **Agent Access Token:** Generated when an agent is deployed. A random 32-character string stored in the agents table. Required to chat with a deployed agent. Can be revoked by the sculptor at any time.
- **Model Provider API Key:** The user's own Claude/GPT API key. Encrypted before storage, decrypted only in memory during proxy calls.

API Key Handling

The platform handles the user's model provider API keys with strict security:

- Encrypted with AES-256-GCM before storage in PostgreSQL

- Decrypted only in memory during Workshop proxy calls, never written to disk
- Never logged in any form — not in error messages, not in request logs, not in Rock Dust
- Never sent to the frontend — the UI shows only the last 4 characters for identification
- Key rotation: users can update their API key at any time; the old key is immediately overwritten, not archived
- If a decryption failure occurs, the agent returns a clear error and the sculptor is prompted to re-enter their key

Rate Limiting

Three layers of rate limiting protect the platform:

- **Per-user limits:** Free tier: 100 requests/hour across all agents. Studio tier: 1,000 requests/hour. Enterprise: configurable. Tracked via Redis using a sliding window counter keyed by user ID.
- **Per-agent limits:** Each sculpted agent has a configurable rate limit (default: 60 requests/minute). Stored in the Sculpture Spec. The Workshop enforces this before making any API call.
- **Per-endpoint limits:** The sculpting endpoints (/api/sculpt/*) are limited to 20 requests/minute per user to prevent abuse of the spec generation flow.

When a rate limit is hit, the API returns HTTP 429 with a Retry-After header. The frontend displays a clear message rather than failing silently.

Input Sanitization

All user input is sanitized before processing:

- **Sculpting messages:** Maximum 2,000 characters per message. Maximum 50 messages per sculpting session. HTML tags are stripped. The conversation is plain text only.
- **Agent chat messages:** Maximum 4,000 characters per message. The Workshop passes user messages to the model API as-is (the model handles interpretation), but the Wall 2 output filter catches any dangerous responses.
- **Sculpture Spec fields:** All spec fields are validated against the JSON schema before storage. String fields have maximum lengths (name: 100 chars, description: 500 chars, system_prompt: 10,000 chars). Arrays have maximum item counts (block_patterns: 50 items, allowed/denied tools: 30 items each).

CORS Policy

The API enforces strict CORS rules:

- Only the platform's own frontend domain is allowed for sculpting endpoints
- Agent chat endpoints allow configurable origins — sculptors can specify which domains are permitted to embed their agent (stored in the Sculpture Spec as an optional allowed_origins array)
- No wildcard origins in production
- Credentials (cookies/auth headers) are only accepted from the platform domain

Agent Isolation

Each sculpted agent operates within its own Sculpture Spec. There is no shared state between agents. The Workshop loads a single spec per request and enforces only that spec's walls. One agent cannot access another agent's tools, data, or conversation history. Agent conversation histories are stored separately and keyed by agent ID plus session ID. There is no mechanism for cross-agent data access.

The Three Walls as Security Layers

- **Wall 1 (Infrastructure) — Strongest:** Tools that don't exist in the API call cannot be invoked. No prompt injection can create a tool that was never offered. This wall has a target enforcement rate of 100%. Any failure here is a critical security bug.
- **Wall 2 (Output) — Strong:** Even if the model generates a violating response, it is caught before reaching the user. V1 uses pattern matching against block_patterns (substring and regex). V1.1 adds AI-powered evaluation using a fast model. Target catch rate: 80%+ in v1, 95%+ in v1.1.
- **Wall 3 (Context) — Weakest alone:** System prompts can be bypassed by sophisticated prompt injection. Specific mitigations: the system prompt includes explicit anti-injection instructions; the prompt states the agent has no awareness of removed capabilities; common jailbreak patterns are addressed in the prompt template. Despite these mitigations, Wall 3 alone is insufficient — it relies on Walls 1 and 2 as safety nets.

The defense-in-depth strategy means that bypassing all three walls simultaneously is extremely difficult. An attacker would need to invoke a tool that was never offered (Wall 1), bypass pattern matching on the output (Wall 2), and override the system prompt (Wall 3) — all in a single request.

Data Privacy

- **Free/Studio tier:** All data routes through our proxy. We log interaction metadata (timestamps, token counts, agent ID) but do NOT log conversation content by default. Users can opt into content logging for debugging. Metadata logs are retained for 90 days, then deleted.
- **Enterprise tier:** The Private Workshop runs in the customer's cloud. No conversation data touches our infrastructure. Only spec metadata and billing data flows through us.
- **Data deletion:** Users can delete their account and all associated data (agents, specs, snapshots, rock dust, sessions) at any time. Deletion is permanent and propagates within 24 hours.

Incident Response

If a wall breach is detected (a carved-away capability is invoked, or a blocked pattern appears in a response that reached the user):

- The affected agent is automatically suspended pending review
- The breach is logged in Rock Dust with full details (request, response, which wall failed, timestamp)
- The sculptor is notified via email
- For Wall 1 breaches (critical): all agents using the same rock/provider are reviewed automatically
- Post-incident: the breach is analyzed, the wall logic is updated, and the fix is verified against the jailbreak test suite before the agent is reinstated

Monitoring

The platform monitors for security-relevant events in production:

- Wall 2 trigger rate per agent — a sudden spike indicates potential attack
- Rate limit hit frequency per user — sustained hitting indicates abuse
- API key decryption failures — indicates key corruption or tampering
- Agent response latency — anomalous latency may indicate proxy issues
- Health check endpoint at /api/health returns status of all services (database, Redis, model API connectivity)

8. Infrastructure

V1 Deployment Architecture

```
Vercel (Frontend)
```

```
    ↓ HTTPS
Railway / Fly.io (Backend + Workshop)
    ↓
PostgreSQL (Neon or Supabase)
    ↓
Redis (Upstash)
    ↓
Claude API (Anthropic)
```

Why These Choices

- **Vercel for frontend:** Zero-config Next.js deployment. Free tier is generous. Scales automatically.
- **Railway/Fly.io for backend:** Simple container deployment. Affordable. Can run both the Sculpting Engine and Workshop as separate services. Easy to scale independently later.
- **Neon/Supabase for Postgres:** Managed PostgreSQL with generous free tiers. Serverless scaling. Built-in connection pooling.
- **Upstash for Redis:** Serverless Redis. Pay-per-request. Perfect for caching specs and rate limiting without running a Redis server.

Cost Estimate (V1, Small Scale)

Component	Service	Monthly Cost
Frontend	Vercel (Pro)	$20
Backend + Workshop	Railway	$10-30
Database	Neon	$0-19
Cache	Upstash Redis	$0-10
Auth	Clerk	$0-25
Domain + DNS	Cloudflare	$0-10
TOTAL		**$30-114/month**

Note: Claude API costs are borne by users (they provide their own API keys). Platform infrastructure cost is independent of model usage.

9. Build Sequence

The MVP should be built in this exact order. Each phase produces a testable, working increment:

Phase 1: Foundation (Week 1-2)

- Set up Next.js project with Tailwind and dual-identity design tokens
- Build the sculpting conversation UI (chat with Sculptor/Rock identity)
- Set up FastAPI backend with Claude API integration
- Define the Sculpture Spec JSON schema and validation
- Build the meta-prompt for the sculpting collaborator
- Test: can a user have a real sculpting conversation with Claude?

Phase 2: The Core (Week 3-4)

- Build the spec generator (conversation history → Sculpture Spec JSON)
- Build the X-Ray visualization component
- Build the Workshop proxy (Wall 1 + Wall 3 enforcement)
- Add Wall 2 pattern-matching filter
- Set up PostgreSQL and create tables
- Test: can a user sculpt an agent and chat with it?

Phase 3: Polish (Week 5-6)

- Build the Chisel Lock flow and Snapshot versioning
- Add Rock Dust logging
- Build Structural Integrity Check
- Add shareable agent links (/agent/[id])
- Build the dashboard (list agents, view specs, view dust)
- Add Clerk auth and user accounts
- Test: full end-to-end sculpting, deploying, and sharing

Phase 4: Launch (Week 7-8)

- Build the landing page with the pitch
- Integrate Stripe for Studio tier payments
- Add token-based usage tracking for free tier limits
- Bug fixes, edge cases, error handling
- Performance optimization (spec caching, response times)
- Deploy to production

- Test: can a stranger sculpt an agent in under 5 minutes?

10. What Is NOT In V1

Explicitly excluded from the MVP to maintain focus:

- Gallery marketplace (no browsing, forking, or selling specs)
- Live collaborative sculpting (single sculptor only)
- Private Workshop / enterprise containerization
- AI-powered Wall 2 interception (pattern matching only)
- Self-sculpting from observation (manual sculpting only)
- Apprentice model (no learning from other sculptures)
- Adaptive carving (no automatic suggestions post-launch)
- Mobile app (web only)
- Multiple model providers in one agent (single rock per agent)

Each of these is a planned future feature with a clear path from v1's architecture. Nothing in v1 needs to be rebuilt to support these — they extend, not replace.

11. Success Metrics

The MVP is successful if:

- **Time to sculpt:** A new user can go from landing page to live agent in under 5 minutes.
- **Sculpt completion rate:** More than 60% of users who start sculpting finish and deploy an agent.
- **Wall 1 effectiveness:** 100% of carved-away tools are inaccessible. Zero exceptions.
- **Wall 2 catch rate:** Pattern matching catches more than 80% of boundary violations in testing.
- **Agent quality:** Sculpted agents score within 5% of manually-configured agents on task completion benchmarks.
- **User sentiment:** Users describe the experience as "creative" or "fun" rather than "technical" or "confusing."

End of Architecture Document

From blueprint to chisel strike.